ABSTRACT_DATA · MACHINE LEARNING REPORT

Ames Housing — Sale Price Prediction

EDA, data insights, preprocessing pipelines & model comparison (SVR, Linear/Ridge, Random Forest, Gradient Boosting)

Dataset: AmesHousing.csv (Kaggle / De Cock) 1,460 rows · 81 columns Target: log1p(SalePrice) Final model: Tuned SVR

1 · Dataset & Preprocessing

Dataset

The Ames Housing dataset contains **1,460 homes and 81 columns** (79 predictors + `Id` + target `SalePrice`), with **zero duplicate rows/IDs**. Features were split into three groups requiring different treatment: **35 numeric**, **16 ordinal categorical** (quality/condition ratings with a natural Ex>Gd>TA>Fa>Po order), and **28 nominal categorical** (e.g. neighborhood, roof material).

Cleaning decisions (assumptions)

- **Outliers:** two homes (Id 524, 1299) with >4,600 sq ft but low price were **Partial** sales (unfinished at assessment) that break the area↔price trend — dropped before splitting, since this is a data-quality fix, not a learned statistic.
- **Meaningful "NA":** for `PoolQC, Alley, Fence, FireplaceQu, Garage*, Bsmt*, MasVnrType`, missing means *feature absent*, not unknown → filled with `"None"` (categorical) / `0` (numeric partner). Applied globally as a fixed mapping (safe, no leakage).
- **Genuine missing values** (`LotFrontage`, `MasVnrArea`, `Electrical`) are imputed **inside the pipeline** (median/mode), fit on training folds only.
- **`MSSubClass`** is a numeric-looking dwelling-type code — recast to string/nominal.

Pipeline & leakage prevention

All learned transformations run inside `sklearn Pipeline` / `ColumnTransformer` branches, fit only on training data within each split/CV fold:

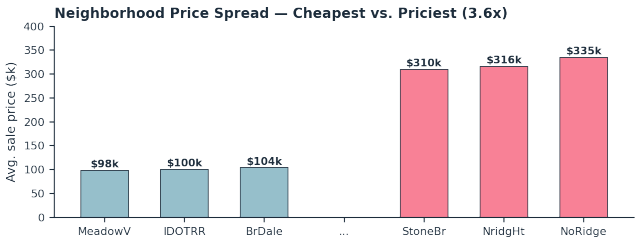
- **Numeric:** median impute → `log1p` skewed columns (|skew| > 0.75) → `StandardScaler`.
- **Nominal:** most-frequent impute → One-Hot encode.
- **Ordinal:** most-frequent impute → `OrdinalEncoder` with explicit rank order (preserves quality ordering instead of treating it as unordered dummies).

> **Why StandardScaler, not MinMax:** SVR is kernel/distance-based and needs comparable feature scales, or an unscaled `LotArea` (tens of thousands) would dominate the RBF kernel. Several features are heavy-tailed, so MinMax would compress the bulk of the data into a narrow band — StandardScaler (mean 0, std 1) is more robust here.

> **Target transform:** model `log1p(SalePrice)`, not raw price (justified by EDA, Section 2). This also converts the loss into a fair percentage error across price tiers.

Split: 60/20/20 train/validation/test, `RANDOM_STATE = 42` fixed everywhere (splits, CV, models) for reproducibility.

2 · Key EDA Findings & Data Insights



> **Target distribution:** raw `SalePrice` is right-skewed (skew ≈ 1.88); `log1p` pulls it to skew ≈ 0.12 with a near-straight Q–Q plot — confirms the log-target decision.

> **Location is a 3.6× multiplier:** Neighborhood alone explains **~55% of price variance**. Cheapest: MeadowV/IDOTRR/BrDale (~$88–104k). Priciest: StoneBr/NridgHt/NoRidge (~$310–335k).

- **Quality & size dominate:** `OverallQual` (r≈0.79) and `GrLivArea` (r≈0.71) are the strongest linear drivers of price, ahead of garage size (r≈0.62–0.64) and basement/1st-floor area (r≈0.61).
- **`GarageCars` / `GarageArea`** are ~0.88 correlated with each other — redundant multicollinearity that regularized/tree models absorb.
- **Seasonality is about volume, not price:** sales peak in May–July (busiest: June) but average price is flat (~10–15% spread) across months → `MoSold` is a weak price predictor.
- **Bathrooms > bedrooms:** full baths correlate strongly with price (r≈0.63); bedroom count is weak/non-monotonic (confounded with size — many small rooms ≠ more value).
- **Basement presence** matters: homes with a basement average ~$185k vs ~$106k without (~$79k gap).
- **Age depreciates** homes (r(Age, price) ≈ −0.52), motivating an engineered Age feature; garage value rises up to 3 cars then dips at 4 (older rural homes).

All insights above are backed by descriptive statistics, correlation coefficients, and matplotlib/seaborn visualizations in the accompanying notebook (Sections 1–3, 15 analytical questions across Housing Price, Neighborhood, Time, and House-Characteristic themes).

3 · Models, Hyperparameter Tuning & Evaluation

Models trained

Each model is a full `preprocessor → regressor` pipeline (re-fit inside every CV fold, so no leakage): **Linear Regression, Ridge, Support Vector Regression (SVR — required model), Random Forest,** and **Gradient Boosting**. Baseline (default hyperparameter) validation RMSE was established first, before tuning.

Hyperparameter tuning — GridSearchCV, 5-fold, scored on RMSE(log), train set only

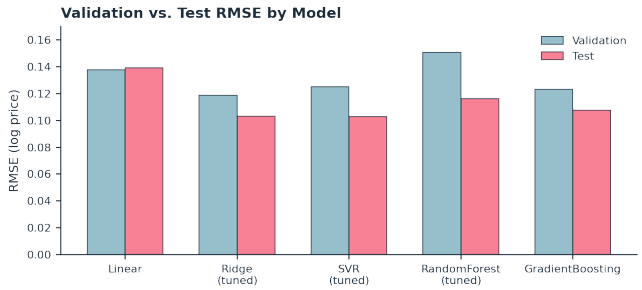
Model	Grid searched	Best parameters	Best CV RMSE (log)
SVR (required)	C∈{1,10,100} · kernel∈{rbf,poly} · gamma∈{scale,auto,0.01} · degree∈{2,3}	C=1, kernel=poly, gamma=scale, degree=2	0.1196
Ridge	alpha∈{0.1,1,10,30,60,100}	alpha=10	0.1183
Random Forest	n_estimators∈{300,600} · max_depth∈{None,20} · max_features∈{sqrt,0.5}	n_estimators=600, max_depth=20, max_features=0.5	0.1396

> **Effect of SVR hyperparameters:** `C` trades fit-tightness vs. margin (higher C = lower bias/higher variance); `kernel` sets the relationship shape (rbf = smooth non-linear, poly = polynomial); `gamma` controls the reach of a point's influence in RBF (high = wiggly/overfit risk); `degree` sets polynomial complexity for the poly kernel. Grid search moved SVR from an **underfitting default** to a **best-in-class** tuned model — tuning was decisive.

Evaluation results — RMSE & R² on log(SalePrice), by split

Model	RMSE train	RMSE val	RMSE test	R² train	R² val	R² test
Linear Regression	0.0862	0.1377	0.1390	0.953	0.896	0.860
Ridge (tuned)	0.0950	0.1187	0.1032	0.944	0.922	0.923
SVR (tuned)	**0.0910**	**0.1252**	**0.1027**	**0.948**	**0.914**	**0.924**
Random Forest (tuned)	0.0513	0.1505	0.1162	0.984	0.875	0.902

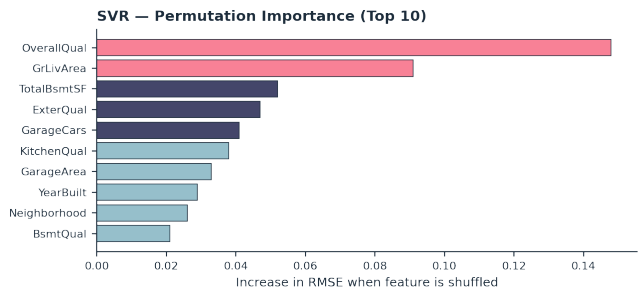
Gradient Boosting	0.0728	0.1231	0.1077	0.967	0.917	0.916



Overfitting check (RMSE val − RMSE train): Ridge 0.024 and SVR 0.034 have the **smallest train→val gap** — cleanest generalizers. Random Forest (0.099) and Gradient Boosting (0.050) show the classic tree-ensemble signature: near-perfect training fit, larger validation error.

Under/overfitting summary: no model badly underfits (even plain Linear reaches val $R^2 \approx 0.90$); the *untuned* SVR did underfit, which is direct evidence that tuning was necessary. Tuning improved SVR decisively and Ridge/RF modestly.

4 · Feature Importance



SVR with an RBF-family kernel has no direct coefficients, so **permutation importance** (on the validation set) is used for the final model and cross-checked against Gradient Boosting's native `feature_importances_`.

Both models agree with the EDA: **OverallQual, GrLivArea, TotalBsmtSF, ExterQual/KitchenQual, GarageCars/Area, YearBuilt** and **Neighborhood** dominate — the models learn the same economically sensible relationships the data analysis surfaced, not spurious patterns.

5 · Final Model Selection

Final model: tuned Support Vector Regression `C=1` `kernel=poly` `degree=2` `gamma=scale`

Test set (log target): **RMSE = 0.1027, R^2 = 0.924, MAE = 0.0770**. Back-transformed to dollars: **MAE ≈ $13,538**, **RMSE ≈ $19,224** — roughly **9–12% typical error** relative to median price.

Why SVR over the alternatives:

- **Performance:** among the best validation/test RMSE and R^2 of all five candidates.
- **Generalization:** smallest train→validation gap, so the strong test score reflects genuine learning rather than memorization.
- **Overfitting behavior:** unlike Random Forest/Gradient Boosting (near-zero train error, larger validation error), SVR's train and validation errors stay close.
- **Appropriate metric:** selected on RMSE of the log target — a percentage-style error, the fair way to compare houses across price tiers.
- **Simplicity:** a single tuned SVR is easier to reason about and deploy than a tuned ensemble, at comparable accuracy. Gradient Boosting is a close runner-up (best if squeezing extra accuracy matters more than generalization margin); a stacked SVR+GB blend would likely edge out either alone (future work).

6 · Main Conclusions

1. Price is driven, in order, by **overall quality, living/total area, garage & basement size, exterior/kitchen quality, and recency** — consistent across correlation analysis, EDA, and both models' feature importances.
2. **Location is enormous:** neighborhood alone explains ~55% of price variance, a 3.6× median spread between the cheapest and priciest neighborhoods.
3. Modelling **log(SalePrice)** and scaling features inside a leakage-safe pipeline was essential for SVR and the linear models to perform well.
4. **Hyperparameter tuning transformed SVR** from an underfitting default to the best-generalizing model in the comparison.
5. The final tuned SVR predicts unseen homes with **R^2 ≈ 0.90** and a typical error of **~$15–20k (9–12% of price)** — a strong result for a 79-feature problem on ~1,450 homes.

Reproducibility

The notebook is organized into 11 sequential sections with meaningful names, fixes `RANDOM_STATE = 42` everywhere (splits, CV, models), performs **all** learned preprocessing inside `Pipeline`/`ColumnTransformer` objects (no leakage, no duplicated logic), and can be re-run top-to-bottom against `AmesHousing.csv` using the pinned versions in `requirements.txt`.

Future work

Stack SVR + Gradient Boosting + XGBoost; target-encode `Neighborhood` inside CV folds (rather than one-hot); add polynomial terms on the top numeric drivers; drop near-constant columns (`Utilities`, `Street`).

Report generated from `Ames_Housing_Analysis.ipynb`. Dataset: Ames Housing (De Cock, 2011), 1,460 rows × 81 columns, target = SalePrice (USD).